

레이다 좌표계와 좌표변환

안테나 극좌표에서 위 · 경 · 고도까지 , 5 단계 변환의 설계와 C 구현 검증

입력 R 20 km · Az 30° · El 0° → 출력 36.233700° N · 127.364345° E · 41.50 m

레이다 시스템 소프트웨어 스텀디

Part2_CoordFrames · CoordCore / radar_coord / CoordUI

좌표계와 좌표변환

안테나 극좌표에서

위 · 경 · 고도까지

01

좌표계 학습

3 - 6 p

과제 1 항 · 6 개 항목 = 좌표계 10 종을 안테나 3 / 플랫폼 4 / 지구 3 으로 정리
결론 — 하나로는 안 된다

02

좌표변환 설계

7 - 9 p

과제 2 항 · 조건 6 개가 지정하는 것 · 설치 기하 · 5 단계 체인과 선정 이유
결론 — 5 단계 6 마디, 전부 정 · 역 쌍

03

C 구현과 검증

10 - 15 p

과제 3 항 · 자료구조와 API · 구현 원칙 · ECEF→LLA 반복 · 실행 결과 · 검증 2 장
결론 — 왕복오차 $1e-10$, 그래도 손검산과 빼보기로 따로 확인

01. 왜 좌표계를 갈아타는가

입력과 출력 사이에는 한 번에 건너뛸 수 없는 어긋남이 셋 있고, 어긋남 하나가 변환 단계를 부릅니다

LIG

어긋남 ① 설치 기하

안테나가 뱃머리를 보지 않고 (선수 기준 시계 90°)
무게중심에도 없다 (뒤 30 m, 위 10 m)

→ 회전 + 평행이동이 필요

어긋남 ② 함선 자세

파도로 배가 매 순간 흔들린다
(이번 입력은 yaw 45°)

→ 자세 회전이 필요

어긋남 ③ 지구 형상

배가 지구 위 한 점 (36.408° N, 127.307° E) 에 얹혀 있고
그 지구는 평면도 구도 아니다

→ 타원체 변환이 필요

안테나 극

→
조건 4

안테나 직교

→
조건 2

동체 (=INS)

→
조건 5

NED

→
조건 6·경유

ECEF

→
조건 6

LLA

조건 3 — INS → 동체 단계 하나 삭제

이 세 어긋남은 뒤에서 숫자로 되돌아옵니다 — ① 30 m · ② 5.9 km · ③ 31.5 m

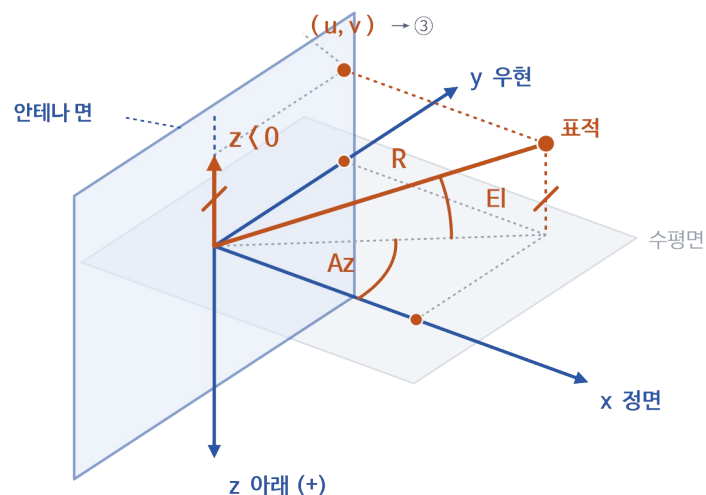
① 의 회전 항은 14p 손검산 (165.000° vs 165.074°) 이, 나머지는 15p 빼보기 실험이 필요합니다

01. 안테나 좌표계 3 종 — Polar · Cartesian · UV

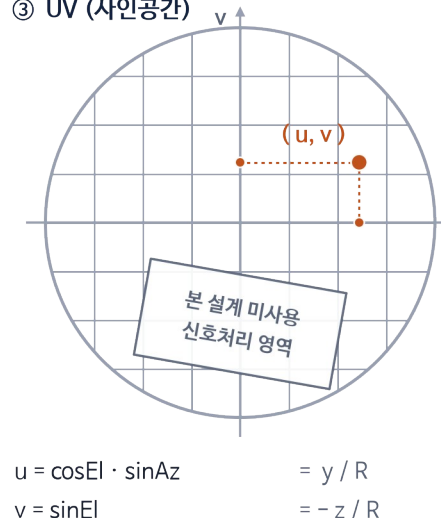
셋은 서로 다른 좌표계가 아니라 같은 방향을 세 가지로 적은 것입니다

LIG

① 극좌표 (R, Az, El) = ② 직교 (x, y, z)



③ UV (사인공간)



표기	성분과 쓰임 · 본 과제 사용 여부
Polar 사용	(R, Az, El) — 신호처리가 내주는 측정 원값. 오차도 이 축 기준. 조건 ④의 입력이자 조건 ⑥의 출력이라 사슬의 양 끝이 이 표기
Cartesian 사용	x 정면 · y 우현 · z 아래 $x = R \cdot \cos El \cdot \cos Az$, $y = R \cdot \cos El \cdot \sin Az$, $z = -R \cdot \sin El$
UV 미사용	$u = \cos El \cdot \sin Az$, $v = \sin El$ — 위상배열 빔 조향 격자 · 안테나 패턴이 이 평면에서 등간격. 신호처리 내부 표기라 처리 사슬 바깥

짚고 가는 것 셋

- 각도끼리는 더할 수 없다 — 회전 · 평행이동을 하려면 직교로 펴야 한다
- z 축은 아래가 + — El > 0 인 표적의 z 는 음수. 이후 모든 좌표계가 이 규약
- 되돌릴 때는 atan2 — asin 은 인자가 1 을 넘으면 NaN, 사분면 판정도 불가

UV 를 안 쓰는 이유는 고정형이라서가 아닙니다 — 신호처리 내부 표기라 처리 사슬 바깥일 뿐입니다

고정면 배열이라도 전자 조향을 하면 UV 는 신호처리 쪽에서 계속 쓰입니다

01. 플랫폼 쪽 — 동체 · INS · NED · ENU

동체는 설치 정보를 흡수하는 자리, INS 는 자세 공급원, NED 는 데이터 처리를 하는 로컬 평면입니다

LIG

좌표계	원점	축 정의	본 과제 사용 여부와 이유
동체 (Body)	함선 무게중심	x 선수 · y 우현 · z 아래 (함선 고정, 자세와 무관)	사용 — 조건 ② 의 설치 정보 (회전 90° + 레버암) 를 여기서 전부 흡수해 이후 단계는 함선만 신경 쓰면 된다
INS	항법장치 기준점	조건 ③ 에 의해 동체와 원점 · 축 일치	사용하되 변환 단계 없음 — roll·pitch·yaw 를 동체→NED 회전에 그대로 대입. 축이 어긋난 함정이면 정렬 회전 1 단계가 추가된다
NED	함선 바로 아래 접평면	x 북 · y 동 · z 아래	사용 — 조건 ⑤ 가 지정한 데이터 처리 좌표계
ENU	같은 접평면	x 동 · y 북 · z 위	미사용 — 앞 두 축을 맞바꾸고 셋째 축 부호를 뒤집은 관계라 둘 다 우수좌표계. 동체 z(아래)·3-2-1 정의에 그대로 이어지는 NED 를 택해 부호 변환을 없앴다

왜 로컬에서 처리하나

위 · 경도에서 직접 추적 필터를 돌리면 경도 1° 의 실제 거리가 위도마다 달라져 등속
직선운동이 직선으로 표현되지 않는다

그 대가

NED 평면은 평평하므로 NED 의 z 를 그대로 고도로 읽으면 안 된다. 등고로 20 km 나간 점의
타원체고는 31.5 m 커진다 → 그래서 ④⑤단계로 ECEF·LLA 를 거친다

조건 ③ 이 변환 단계 하나를 지우고, 조건 ⑤ 가 처리 좌표계를 NED 로 고정합니다

01. 지구 쪽 — ECEF · ECI · LLA

여기서부터는 지구 모양 (WGS-84) 이 계산에 직접 개입하고, 셋 중 하나는 이번 설계에 필요가 없습니다

LIG

좌표계	원점 / 축	본 과제 사용 여부와 이유
ECEF	지구 중심 · 지구와 함께 회전 X 그리니치 적도, Z 자전축	경유로 사용 — 국소 평면을 지구 전역 좌표로 올리는 유일한 다리. 원점이 플랫폼에 묶이지 않아 타 플랫폼 · 전시기와 같은 좌표로 대화 가능
ECI	같은 원점 · 별 기준 관성 고정 자전축 둘레 회전 하나 차이	미사용 — 사슬 전체가 지구고정계에서 닫히고 관성 전파 · 천측이 없다. 위성 · 궤도 연동 시에만 한 단계 추가 (그때는 시각 동기가 정밀도를 지배)
LLA	WGS-84 타원체 기준 위도 · 경도 · 타원체고	사용 — 조건 ⑥이 요구하는 최종 출력 형식이자 사람과 전시기가 읽는 형식

WGS-84 — 실제 계산에 쓰인 값

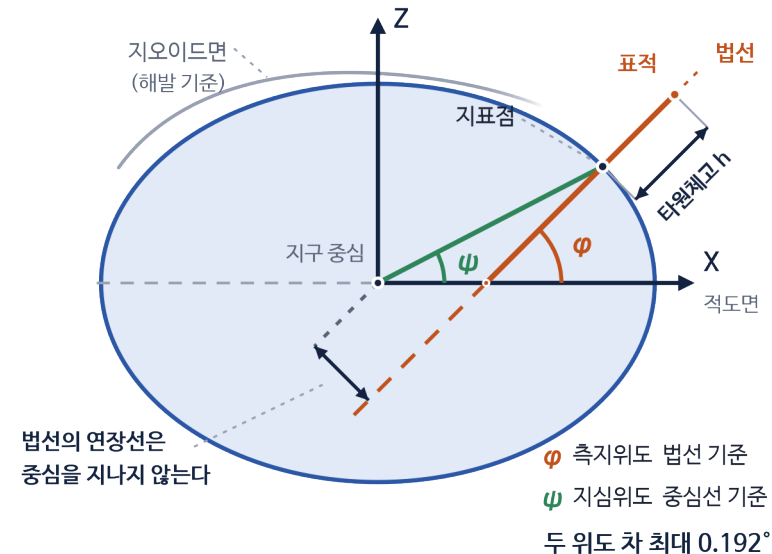
$$a = 6,378,137.0 \text{ m} \quad e^2 = 0.00669438 \quad N(36.408^\circ) = 6,385,671.083 \text{ m}$$

타원체고 ≠ 해발 — 한국 근해 지오이드고가 약 +20 ~ +25 m 이므로

타원체고 41.5 m 는 해발로는 약 17 ~ 22 m 다

타원체면의 법선

측지위도 · 지심위도 · 타원체고 (편평률 과장)



타원체면의 법선이 지구 중심을 지나지 않습니다 — 그래서 측지위도와 지심위도가 갈립니다

이 하나가 12p 의 ECEF→LLA 가 한 번에 안 풀리는 이유이기도 합니다

02. 조건 6 개가 지정하는 것

조건은 배경 설명이 아니라, 어떤 좌표계를 쓰고 어떤 항을 넣을지 하나씩 못 박은 설계 사양입니다

LIG

조건	내용	설계에 지정하는 것
① 플랫폼	해상 함선 + 고정형 안테나	안테나 지향이 상수라 짐벌 단계 없음. 대신 파도로 자세가 매 순간 변해 자세 변환이 필수
② 설치	선수 기준 시계 90° 무게중심 뒤 30 m · 위 10 m	안테나→동체에 회전 Rz(90°) 와 레버암 (-30, 0, -10) m 두 항이 모두 필요 (회전만으로는 불가)
③ INS	무게중심과 축이 일치	INS 좌표계 = 동체 좌표계 → 변환 단계 하나를 지운다 (축이 어긋나면 정렬 회전이 하나 더 붙는다)
④ 입력	안테나 기준 R · Az · El	사슬의 시작점이 안테나 극좌표로 확정, 첫 단계는 극 → 직교
⑤ 처리	데이터 처리는 로컬에서	NED 를 처리 좌표계로 확정 — 여기서 멈추라는 조건이지 ECEF 로 올라가라는 조건이 아니다
⑥ 출력	LLA + 안테나 극좌표	ECEF 경유와 LLA 도달을 요구하는 유일한 조건이자, 모든 변환을 정 · 역 쌍으로 만드는 조건

선택 5 마디 안테나 극 · 안테나 직교 · 동체 (=INS) · NED · LLA **경유 1 마디** ECEF **제외 3** UV · ECI · ENU

조건 ③⑤⑥ 이 각각 단계 하나를 지우고, 하나를 고정하고, 함수 개수를 두 배로 만듭니다

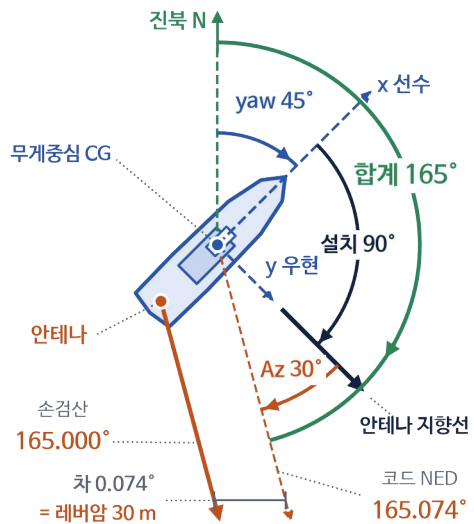
02. 조건 ②의 기하 — 설치 방위와 레버암

방위각은 선수각 · 설치각 · 측정각이 더해져 만들어지고, 레버암은 거리가 멀어져도 줄지 않습니다

LIG

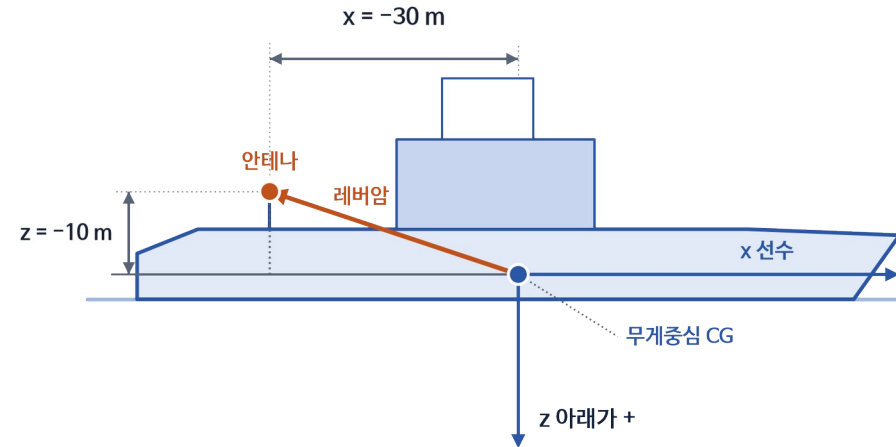
평면도 — 진북 기준 방위의 누적

위쪽이 진북 · 각도는 시계방향(+)



측면도 — 레버암 (-30, 0, -10) m

무게중심이 원점 · x 선수 · z 아래가 +



진북 기준 방위 = 세 각의 합

yaw 45° + 설치 90° + Az 30°

= 165.000°

손검산 — 레버암 0 · 안테나 보어사이트

코드 NED 165.074°

NED 원점 (무게중심) 에서 본 방위

차이 0.074° 는 오차가 아니다.

두 기준점이 30 m 떨어져 있기 때문 (20 km 에서 약 26 m)

레버암은 평행이동

표적이 20 km 든 100 km 든 같은 30 m · 10 m 가 남는다.

거리에 희석되는 각도 오차와 구분되는 서명 (15p A)

z 축이 아래가 + 라서 '뒤 30 m · 위 10 m' 는 (-30, 0, -10) 이 됩니다

02. 5 단계 변환 체인과 선정 이유

좌표변환의 재료는 회전과 평행이동 둘뿐이고, 조건 ⑥ 때문에 다섯 단계 전부에 역방향을 둡니다

LIG

읽는 법 $p_B = C(B \leftarrow A) \cdot p_A + t$ 원점이 같으면 $t = 0$ 으로 회전만, 원점이 다르면 t 가 붙는다

단계	from → to	회전 C	평행이동 t	원점	이 단계를 두는 이유
①	극 → 안테나 직교	— (삼각함수)	—	동일	각도끼리는 더할 수 없어 벡터로 펴야 회전 · 이동이 가능 (조건 ④)
②	안테나 → 동체	Rz(설치방위) · Ry(백틸트)	+ 레버암 (-30, 0, -10)	다름	안테나 설치 정보를 여기서 전부 흡수 (조건 ②)
③	동체 → NED	Rz(yaw) · Ry(pitch) · Rx(roll) [3-2-1]	—	동일	INS 자세를 반영해 흔들리는 축을 북 · 동 · 하로 고정 (조건 ⑤ 도달)
④	NED → ECEF	Rz(경도) · Ry(-90° - 위도)	+ 함선 ECEF 위치	다름	국소 평면을 지구 전역 좌표로 올림 (조건 ⑥ 이 요구하는 경우)
⑤	ECEF → LLA	회전 아님 (WGS-84 반복해)	—	—	전시기가 읽는 형식으로 (조건 ⑥) — 유일하게 한 번에 안 풀린다

역방향 회전은 전치, 평행이동은 빼기, 순서는 거꾸로 — 같은 회전행렬을 재사용하므로 함수가 10 개여도 실제 로직은 5 쌍

역변환은 값을 되돌리는 검사가 아니라 실제 운용 경로입니다

LLA 는 전시기 · 상급부대가 공유하는 절대 좌표, 안테나 극좌표는 추적 게이트 · 빔 재지향에 쓰는 상대 좌표

03. 구현 구조 — 자료구조와 API

변환 로직을 라이브러리 한 곳에 몰고 함수 이름에 방향을 박아, 호출부에서 헛갈릴 여지를 없앴습니다

LIG

모듈 구성 — 로직은 한 곳에만

CoordCore 정적 라이브러리. coord_frames.c/h + 지금 matrixCalcLib.c/h

radar_coord 콘솔. 과제 3.1 입력으로 단계별 값 · 왕복오차 · 최종출력 출력

CoordUI MFC 다이얼로그. 변환 로직 없이 f_* 호출만, 슬라이더로 자세 변경

→ 변환 로직이 한 곳에만 있으므로 검증 대상이 하나로 고정된다

API 10 개 = 정 · 역 5 쌍

f_PolarToXyz ↔ f_XyzToPolar

f_AntToBody ↔ f_BodyToAnt

f_BodyToNed ↔ f_NedToBody

f_NedToEcef ↔ f_EcefToNed

f_LlaToEcef ↔ f_EcefToLla

짝이 없는 함수가 있으면 그 자체가 설계가 덜 끝났다는 신호

coord_frames.h

```
// 좌표계마다 타입을 따로 둔다 — 잘못 넣으면 컴파일러가 잡는다
typedef struct { FLOAT64 dRange, dAz, dEl; } ST_Polar;
typedef struct { FLOAT64 dLat, dLon, dAlt; } ST_Lla;
typedef struct { FLOAT64 dRoll, dPitch, dYaw; } ST_Attitude;
typedef struct { FLOAT64 dAz, dTilt; ST_Vec3 stOffset; } ST_Mount;
```

main.c — 조건이 코드가 되는 지점

```
// 조건 ② 가 상수 하나로 그대로 들어온다 ( 고정형이라 런타임에 안 바뀜 )
ST_Mount stMount = { DEG2RAD(90.0), DEG2RAD(0.0),
                    { -30.0, 0.0, -10.0 } };
```

지금 라이브러리 함정

Matrix_Identity 는 9×9 기준이라 3×3 에 못 쓴다
→ Matrix_initialize(&m, 3, 3) 으로 0 을 채운 뒤 대입

사내 규칙 그대로 — ST_ 구조체 · f_ 함수 · FLOAT64 / INT32 / VOID 재정의형

단위는 라이브러리 안에서 rad·m 로 통일하고 deg 변환은 경계 (main, UI) 에서만 합니다

03. 구현 원칙 — 회전은 곱으로만, 역은 전치로만

행렬 원소를 손으로 쓰지 않으면 틀릴 수 있는 곳이 세 함수로 줄어듭니다

LIG

coord_frames.c — 회전은 축 회전 곱으로만

```
// 3-2-1 순서를 이 함수 하나에 가둔다
static matrix f_RotBodyToNed(const ST_Attitude stAtt)
{
    matrix stRz = f_RotZ(stAtt.dYaw);
    matrix stRy = f_RotY(stAtt.dPitch);
    matrix stRx = f_RotX(stAtt.dRoll);

    return Matrix_Product3(&stRz, &stRy, &stRx);
}
```

coord_frames.c — 역은 전치로만

```
// 역변환에 새 행렬을 만들지 않는다. Matrix_Inverse 도 쓰지 않는다
ST_Vec3 f_NedToBody(const ST_Attitude stAtt, const ST_Vec3 stNed)
{
    matrix stR = f_RotBodyToNed(stAtt); // 정변환과 같은 함수
    matrix stRt = Matrix_Transpose(&stR);
    matrix stOut = Matrix_Product2(&stRt, &stP);

    return f_ToVec(&stOut);
}
```

1 이름에 방향을 박는다

f_AtoB / f_BtoA

정 · 역이 같은 회전행렬 함수를 공유하므로 한쪽만 고치는 실수가 구조적으로 불가능하다.

틀릴 수 있는 지점이 f_RotAntToBody · f_RotBodyToNed · f_RotNedToEcef
세 함수로 줄어든다

2 회전은 곱으로만, 역은 전치로만

3×3 원소 9 개를 손으로 쓰면 부호 하나 틀려도 눈으로 못 잡는다.

직교행렬이라 전치가 곧 역이고, 일반 역행렬보다 정확하고 빠르다

회전은 좌표축을 바꾸는 것이지 점을 옮기는 것이 아니다

단위는 라이브러리 안에서 rad·m 만 씁니다 — deg 가 중간에 섞이면 값이 그럴듯해서 더 못 잡습니다

03. ECEF → LLA — 유일하게 한 번에 안 풀리는 단계

위도를 알아야 N 이 나오고 N 을 알아야 위도가 나오는 순환이라 반복으로 풀되, 반드시 상한을 둡니다

LIG

타원체면의 법선이 지구 중심을 지나지 않기 때문에 (6p) 위도와 고도가 서로를 참조한다 — 나머지 네 단계는 전부 닫힌 식이지만 이 단계만 예외

coord_frames.c — f_EcefToLla

```
dLat0 = atan2(stEcef.dZ, dP); dAlt0 = 0.0; // 구로 본 위도에서 출발
for (i = 0; i < LLA_ITER_MAX; i++)
{
    dN = WGS84_A / sqrt(1.0 - WGS84_E2 * sin(dLat0)*sin(dLat0));
    dAlt = dP / cos(dLat0) - dN;
    dLat = atan2(stEcef.dZ * (dN + dAlt),
                 dP * (dN * (1.0 - WGS84_E2) + dAlt));

    if ((fabs(dLat - dLat0) < LLA_TOL_LAT) &&
        (fabs(dAlt - dAlt0) < LLA_TOL_ALT)) break;
    dLat0 = dLat; dAlt0 = dAlt;
}
```

1 회가 - 14,795 m 까지 튀는 이유

초기 위도를 지심위도로 잡은 대가다. 6p 의 측지 - 지심 위도차 0.192° 가 그 정체 — 위도 오차 0.18° 가 고도 오차 14.8 km 로 증폭된다. 3 회면 m, 6 회면 μm 단위로 수렴

실제 수렴 과정 (고도 h)

회	고도 h [m]	회	고도 h [m]
1	-14,795.626	4	41.4954
2	76.360	5	41.49520
3	41.413	6	41.495196

상수

LLA_ITER_MAX 10 반복 상한

LLA_TOL_LAT $1\text{e-}12$ rad LLA_TOL_ALT $1\text{e-}6$ m 종료 허용오차

그럼에도 상한을 두는 이유

실시간 태스크에서 무한루프는 오차보다 위험하다. 종료 조건은 위도와 고도 두 개를 동시에 만족해야 통과

다섯 단계 중 넷은 닫힌 식이고, 반복이 필요한 것은 이 한 단계뿐입니다

03. 실행 결과 — 단계별 값과 최종 출력

과제 입력값 한 세트가 다섯 단계를 지나 위 · 경 · 고도로 나옵니다

LIG

입력 R 20,000 m · Az 30.000° · El 0.000° **함선** 36.408° N, 127.307° E, 0 m · roll 0°, pitch 0°, yaw 45° **설치** 방위 90°, 백틸트 0°, 레버암 (-30, 0, -10) m

단계	결과	눈으로 하는 확인
① 극 → 직교	(+17,320.5081, +10,000.0000, -0.0000) m	
② → 동체	(-10,030.0000, +17,320.5081, -10.0000) m	Rz(90°) 가 안테나 x 를 동체 y 로 그대로 보냈고, x 성분 - 10,000 에 레버암 - 30 이 더해져 - 10,030
③ → NED	(-19,339.7297, +5,155.1677, -10.0000) m	진북 기준 165.074°, 지상거리 20,015.017 m
④ → ECEF	표적 (-3,125,893, +4,093,775, +3,749,164) m 함선 (-3,114,830, +4,087,763, +3,764,723) m	두 점의 차 크기 $\approx$ 20,015 m 로 ③ 의 지상거리와 일치 (자릿수는 표시용으로 자름)
⑤ → LLA	36.233700252° N, 127.364344961° E, 41.4952 m	반복 6 회 수렴

① 표적 위 / 경 / 고도

36.233700° N, 127.364345° E, 41.50 m

② 표적 안테나 극좌표

R 20,000.000 m, Az 30.000°, El 0.000°

최종값만 찍지 않고 단계별 중간값을 전부 출력하게 만들어, 틀렸을 때 어느 단계인지 바로 보이게 했습니다

03. 검증 ① 왕복오차와 손검산 — 잡는 오류가 다르다

왕복은 정·역의 비대칭을 잡고, 손검산은 회전 방향·순서 자체를 잡습니다

LIG

두 검증은 잡는 오류의 종류가 다르므로 하나로 다른 하나를 대신할 수 없다

① 왕복 검증 — 정·역의 비대칭을 잡는다

표적 LLA → ECEF → NED → 동체 → 안테나 직교 → 극좌표로 되짚어 입력과 비교

R 20,000.000000 m

Az 30.000000°

El 0.000000°

왕복오차 dR $3.6e-10$ m · dAz $2.1e-12$ ° · dEl $2.5e-12$ °

→ 배정밀도 반올림 수준

잡히는 오류

정·역 쌍의 부호 실수 · 전치 빠짐 · 평행이동 순서 뒤바뀜

한계 — 정변환과 역변환이 같은 방향으로 함께 틀리면 왕복은 그대로 통과한다

② 손검산 — 회전 방향·순서 자체를 잡는다

손검산 선수 45° + 설치 90° + 측정 30° = 165.000°

레버암 0 가정 · 안테나 보어사이트에서 잰 값

코드 NED 결과 = 165.074°

NED 원점 (무게중심) 에서 본 표적 방위

차이 0.074° 는 오차가 아니다. 안테나가 무게중심보다 30 m 뒤에 있어 두 기준점에서 본 방위가
달리다 — 8p 평면도의 두 시선

이 검산식 자체는 배가 수평이고 레버암이 0 일 때만 성립하는 근사다

잡히는 오류

Rz 회전 방향 (시계 / 반시계) 과 3-2-1 곱 순서가 통째로 틀리는 경우 — 3p 어긋남 ① 의 회전
향을 검증하는 자리

두 검증 모두 radar_coord 가 실행할 때마다 스스로 출력하게 만들어 두면 그대로 회귀 검사가 됩니다

03. 검증 ② 항을 빼 보기, 그리고 평면 근사의 해석

항을 하나씩 빼 보면 그 항이 왜 있는지가 숫자로 드러납니다

LIG

3p 의 세 어긋남 — ① 설치 기하 / ② 함선 자세 / ③ 지구 형상 — 이 여기서 30 m · 5.9 km · 31.5 m 로 되돌아온다 (① 의 회전 항은 14p 손검산이 맞는다)

A 레버암 생략 — 어긋남 ① 설치 기하

수평 30 m, 고도 10 m 가 그대로 어긋난다

표적이 멀어져도 줄지 않는다

각도 오차가 아니라 위치 오차라서 거리에 희석되지 않는 것이 이 항의 서명

B roll 무시 — 어긋남 ② 함선 자세

roll $0^\circ \rightarrow$ 실제 고각 $+0.03^\circ$ $10^\circ \rightarrow -8.61^\circ$ $20^\circ \rightarrow -17.19^\circ$

roll 20° 면 20 km 앞 표적 고도가 약 5.9 km 어긋난다

→ 자세 보정은 옵션이 아니다

C 평면 근사와 타원체의 차이 — 어긋남 ③ 지구 형상 (실험이 아니라 정상 결과의 해석)

로컬 평면을 고도 기준면으로 착각했을 때, 즉 NED 의 $z(-10\text{ m})$ 를 그대로 고도로 읽었을 때 생기는 값

5 km	2.0 m	10 km	7.9 m	20 km	31.5 m	50 km	196.7 m	100 km	786.4 m
------	-------	-------	-------	-------	--------	-------	---------	--------	---------

이 값은 구현의 오차가 아니라 (왕복오차 $1e-10$ 이 그 증거) 사슬이 ④⑤단계로 ECEF·LLA 를 거치는 이유다

표적 타원체고 41.50 m = 안테나 높이 10 m + 20 km 에서의 평면 - 타원체 차 31.5 m

13p ⑤ 단계 출력이 우연이 아님을 손으로 확인합니다

정리와 남은 것

좌표변환은 회전과 평행이동의 조합일 뿐이고, 어려운 것은 수식이 아니라 규약을 끝까지 지키는 일입니다

LIG

과제 1 좌표계 학습

6 개 항목 10 종을 안테나 / 플랫폼 / 지구 세 묶음으로 정리했다. 좌표계를 정하는 것은 원점과 축 둘뿐이고, 그 둘이 그대로 평행이동과 회전이 된다

과제 2 좌표변환 설계

조건이 좌표계를 정한다 — ③ 이 단계를 지우고, ⑤ 가 NED 를 고정하고, ⑥ 이 ECEF 경유와 모든 함수의 정 · 역 쌍을 만들었다. UV · ECI · ENU 를 쓰지 않는 이유까지 조건과 규약으로 설명된다

과제 3 C 구현과 검증

정 · 역 10 개 함수로 $36.233700^{\circ} \text{ N}$ / $127.364345^{\circ} \text{ E}$ / 41.50 m 를 얻었고, 왕복오차로 끝내지 않고 손검산과 빼보기 실험으로 각 항이 실제로 일하는지 따로 확인했다

남은 것

지구고정계 밖으로 나갈 때

ECEF ↔ ECI 는 자전축 둘레 회전 한 단계 차이. 위성 · 궤도 연동 시 시각 동기가 정밀도를 지배 ($1 \text{ ms} =$ 적도 0.47 m)

타원체고 → 해발고도

지오이드 모델이 필요하다. 한국 근해 $+20 \sim 25 \text{ m}$ 라 41.5 m 는 해발 약 $17 \sim 22 \text{ m}$ — 전시 규격 확인 필요

회전형 안테나로 바뀌면

짐벌각 한 단계가 안테나 ↔ 동체 사이에 추가된다. 나머지 체인은 그대로다

부호 · 축 · 단위 규약을 헤더 주석에 못 박아 두는 것이 가장 값싼 방어입니다